

FEDERAL AUTONOMOUS EDUCATIONAL
INSTITUTION OF HIGHER PROFESSIONAL EDUCATION
«NATIONAL RESEARCH UNIVERSITY
«HIGHER SCHOOL OF ECONOMICS»

Faculty of Informatics, Mathematics, and Computer Science

Novikova Irina Alekseevna

**DEVELOPMENT OF A NEW METHOD FOR LLM FINE-TUNING BASED
ON IMPORTANCE OF BUILDING BLOCKS**

Diploma

in the field of 01.04.02 Applied Mathematics and Informatics

educational program

"Data Mining"

Reviewer

Visiting Lecturer

A.A. Shimko

Academic Supervisor

PhD in Computer Science, Associate

Professor

A.V. Demidovskij

Nizhny Novgorod, 2026

Contents

Introduction	4
Chapter 1. Theoretical foundations of fine-tuning	
acceleration through importance analysis	7
1.1 Definition of Transformer-based architecture	7
1.2 Key directions in Large Language Models fine-tuning	
acceleration	8
1.2.1 Data reduction	9
1.2.2 Quantization	9
1.2.3 Pruning	10
1.2.4 Backpropagation elimination	11
1.2.5 Parameter-Efficient Fine-Tuning	12
1.2.6 Summary	14
1.3 Existing approaches for assessing the importance within	
Large Language Model	15
1.3.1 Importance of separate weights	15
1.3.2 Importance of layers	19
1.3.3 Importance of Transformer blocks	20
1.3.4 Neural Architecture Search	22
1.3.5 Summary	23
Chapter 2. Development of an approach to accelerate the fine-tuning through	
importance analysis	25
2.1 Importance-based Transformer blocks selection	27
2.1.1 Statistics collection phase	27
2.1.2 Selection phase	27
2.2 Key differences from existing approaches	28

Chapter 3. Evaluation of the proposed method	30
3.1 Software implementation	30
3.2 Protocol	30
3.3 Analysis of the obtained results	32
3.4 Sensitivity analysis	33
3.4.1 Analysis of hyperparameter α	33
3.4.2 Analysis of hyperparameter h	34
3.4.3 Analysis of importance metrics	35
3.5 Summary	36
Conclusion	37
3.1 Results	37
3.2 Relevance	38
3.3 Future directions	39
Reference List	40

Introduction

The **relevance** of this research lies in the central function that Large Language Models (LLMs) now serve across a range of fields, such as machine learning [1], computer vision [2], and natural language processing [3]. These advanced models, trained on vast datasets, have become foundational components in various commercial and industrial settings, valued for their strong performance in activities like in-context learning [3], following complex instructions [4], and engaging in structured reasoning and reasoning. Nevertheless, the capabilities of generic, pre-trained LLMs frequently fall short when it comes to highly specialized applications, such as those found in medical or scientific contexts. To bridge this gap, models often undergo a process called fine-tuning, which adapts them to specific downstream tasks. Unfortunately, this adaptation is highly demanding in terms of both computational resources and time. As LLMs have increased in size and complexity, the costs associated with training – both financial and environmental – have surged accordingly. Additionally, the energy consumed by the latest generation of LLMs has reached unprecedented levels, raising further concerns about sustainability. For instance, fine-tuning of the DeepSeek-V3 model requires about 3M H800 GPU hours [5] and approximately 597 tons of carbon emissions [6].

Given these challenges, there is a pressing need for innovative fine-tuning techniques that can reduce resource consumption without compromising model efficacy. Developing such methods is critical for ensuring that advances in LLM technology remain accessible and cost-effective.

At the same time, the enormous parameter counts in today’s LLMs result in significant redundancy, with many parameters contributing little to overall performance [7]. Thus, identifying and making use of the most influential components of these models is essential [8]. The effectiveness of LLMs – whether during pre-training, fine-tuning, or inference – often hinges on recognizing which elements of the model architecture are most impactful. This insight is foundational not only for model opti-

mization strategies like Neural Architecture Search [9], [10], [11], quantization [12], [13], [14], and pruning [15], [16], [17], but also for accelerating and streamlining the fine-tuning process itself. However, existing approaches to measuring the importance of model structures frequently rely on disparate heuristics, many of which do not have a clear or consistent relationship with actual model performance [9].

Therefore, there are numerous avenues for optimizing the use of LLMs, particularly through applying methodologies for assessing importance of structures within LLMs. However, to date, there have been no comprehensive studies that systematically investigate methods for importance assessment across various structural levels. Addressing this gap is not only crucial for enhancing model efficiency but also for advancing the methodologies used in model optimization, in particular, in the field of fine-tuning acceleration.

The **practical significance** of this research is that it introduces an innovative fine-tuning strategy for LLMs, which provides up to 44.8% of acceleration compared to state-of-the-art *LoRA* approach, thus enabling the further deployment of LLMs in resource-constrained environments.

The **novelty** of this work can be summarized as follows:

- This work introduces a comprehensive overview of the most prominent existing methodologies for importance estimation of different structures within LLMs.
- This study introduces a new fine-tuning approach for LLMs, that dynamically assigns *LoRA* adapters during the fine-tuning based on importance analysis.

The **goal** of the present study is to develop a method for accelerating the fine-tuning of LLM while maintaining quality, with performance degradation compared to the baseline *LoRA* approach being less than 1%.

To achieve the goal, the following **objectives** were formulated:

1. Conduct a comprehensive background study on the topic of fine-tuning acceleration techniques and existing methodologies for estimating the importance of various structures within LLM.

2. Propose a novel *LoRA*-based fine-tuning strategy based on importance analysis.
3. Offer a detailed evaluation of the proposed method and its hyperparameters among various LLMs.
4. Conduct a comparative analysis between the proposed solution and the closest competitors in the field.
5. Analyze obtained results and draw conclusion.

The study employs a combination of **methodological approaches** to address the research objectives. Initially, it conducts a theoretical review and synthesis of relevant academic works, providing a strong conceptual basis for the investigation. Next, programming techniques are applied to evaluate the significance of LLMs architectures. Finally, Deep Learning methodologies are used to further refine the performance of LLMs.

The **object** of this study is the fine-tuning acceleration of LLMs through importance analysis of Transformer blocks.

This study was conducted as part of a **research project at the Huawei** aimed at accelerating the fine-tuning of LLMs with Parameter-Efficient approaches. The results of this research were presented at the European Conference on Artificial Intelligence 2025 (ECAI, Rank A), Neuroinformatics 2024, and published in the articles in journals indexed in Scopus (Q4, Q3). **An individual contribution** include the development of the key idea of method, which suggests to perform an importance analysis during the *LoRA*-based fine-tuning and attach *LoRA* adapters only to the critical Transformer blocks. In addition, an overview of existing competitors in the field was performed as well as experiments on applying proposed approach on a wide range of LLMs with following analysis of obtained results.

Chapter 1. Theoretical foundations of fine-tuning acceleration through importance analysis

The goal of the present study is to develop a method for accelerating the fine-tuning of Large Language Models. However, the development of a novel approach requires a comprehensive analysis of existing techniques and promising ideas in this field, which can be leveraged in future work. This chapter is dedicated to such a thorough analysis and review.

1.1 Definition of Transformer-based architecture

Let $\mathcal{M}_\Phi$ denote a Transformer autoregressive model with trainable parameters $\Phi \in \mathbb{R}^n$. The model $\mathcal{M}_\Phi$ is described with a set of Transformer blocks τ , where each block τ_i contains two submodules: a multi-head attention module (MHA) (1) and a module consisting of a Feed Forward Neural Network (FFNN) (2). Given the input sequence $x \in \mathbb{R}^n$, MHA module works with h parallel heads.

$$head_i = Softmax\left(\frac{xW_q(xW_k)^T}{\sqrt{d_h}}\right)xW_v \quad (1)$$

$$MHA(X) = Concat(head_1, \dots, head_h)W_o,$$

where $i \in \mathbb{R}$, $W_o \in \mathbb{R}^{d \times d}$ is an output projection, and $W_q, W_k, W_v \in \mathbb{R}^{d \times d_h}$ are query, key, and value projections of the i -th head; d_h is typically set to d/h . FFNN module consists of two linear transformations with an activation function (e.g. ReLU) (2).

$$FFNN(x) = ReLU(xW_{up} + b_{up})W_{down} + b_{down}, \quad (2)$$

where $W_{up} \in \mathbb{R}^{d \times d}$ is the up-projection matrix and $W_{down} \in \mathbb{R}^{d \times d}$ is the down-projection matrix.

Within the LLM there exist a concept of the "importance" of certain structures. In this context, "importance" facilitates the identification of structures that can be

removed without affecting model performance. Thus, understanding the importance of a structure helps assess its impact on the model’s overall quality.

Fine-tuning is the process of turning a general-purpose model into a specialized one while updating all pre-trained model parameters over a downstream task dataset. During the fine-tuning process LLM parameters are updated until the loss function reaches approximately the optimum, indicating, that the model has been converged [18] and is ready for deployment.

Let fine-tuning data be organized as a dataset $D = \{(x_i, y_i), i = 1, \dots, N\}$, where x_i is a data sample, and y_i is a ground truth label; both x_i, y_i are sequences of tokens. Model prediction is denoted as $\mathcal{M}_\Phi(x_i)$. Fine-tuning assumes that the model is pre-trained: let it be initialized with pre-trained weights Φ_0 .

Full fine-tuning is the process of updating pre-trained weights to $\Phi_0 + \Delta\Phi$ so that the fine-tuning objective for $\mathcal{M}_\Phi$ is optimized over Φ (3).

$$\max_{\Phi} \sum_{(x,y) \in D} \sum_{t=1}^{|y|} \log(\mathcal{M}_\Phi(y_t|x, y_{<t})), \quad (3)$$

where $|y|$ is the number of tokens in the data sample, dimensionality of $|\Delta\Phi|$ is equal to $|\Phi_0|$ [19].

The resources required for full fine-tuning of a model are comparable to those needed for pre-training an LLM; the only difference is that the full fine-tuning necessitates a smaller amount of a domain-specific data. Therefore, the development of approaches, that allow to reduce the number of required resources is of crucial importance.

1.2 Key directions in Large Language Models fine-tuning acceleration

Various modern approaches exist for reducing time and memory overhead during the full fine-tuning, including data reduction [20], quantization [21], pruning [22], backpropagation elimination [23], [24], and Parameter-Efficient Fine-Tuning [19].

1.2.1 Data reduction

Data reduction is the process of selectively removing less significant samples from a training dataset while maintaining the accuracy of the resultant model. This process encompasses three primary methodologies: dataset condensation or distillation, coreset selection, and importance sampling, each defined by its approach to create a compact version of the training dataset.

Dataset distillation involves synthesizing new samples based on statistical properties derived from the original dataset [25]. This method aims to condense the dataset into a smaller, yet representative, set of synthetic samples which effectively capture the underlying data distribution. Coreset selection facilitates the automatic identification and extraction of a subset from the original dataset [26]. This approach ensures that the core elements necessary for learning are retained, thereby optimizing the training process. Importance sampling focuses on selecting samples that contribute most significantly to the model’s learning, particularly those that induce larger updates to the model’s weights during training [27]. For example, ALOE [20] removes data samples based on losses obtained from a currently trained model or a pre-trained one, providing up to 92% of acceleration with quality degradation of 1.5% compared with *LoRA*. Moreover, these methods cause an additional overhead to the training process by requiring a full forward pass for all samples in a dataset on importance estimation [28].

1.2.2 Quantization

Quantization is a technique used to transform high-precision data into low-precision data. This process is predominantly utilized to accelerate inference, but recent studies have also explored its application in enhancing the efficiency of the fine-tuning.

One notable approach is QLoRA [21], which introduces a 4-bit NormalFloat data type for storing frozen pre-trained language model while maintaining computational

operations, such as matrix multiplication of *LoRA* adapters, in 16-bit BrainFloat precision. Despite achieving a substantial memory reduction, this method provides slow-down compared to standard *LoRA* approach due to quantization and dequantization of LLM weights. In addition, when applying the extreme low-bit quantization, the huge quantization error can adversely impact the initialization of *LoRA* fine-tuning [29]. Another method, PEQA [30], focuses on fine-tuning the quantization scales of quantized LLMs, while keeping the integer matrices unchanged. This approach results in up to 92% memory reduction, however introduces a meaningful quality degradation of 5% compared to *LoRA*. Additionally, AlphaTuning is a technique that involves quantizing a pre-trained LLM and fine-tuning certain quantized parameters for specific tasks [31]. This method not only accelerates fine-tuning by 38% but also reduces memory usage by 80% with a quality decrease of at least 5%.

1.2.3 Pruning

Model pruning is a technique aimed at reducing the model by eliminating redundant parameters or substructures. This process is primarily categorized into three approaches: unstructured, semi-structured, and structured pruning [22].

Unstructured pruning removes individual parameters based on importance criterion, achieving high theoretical sparsity and minimal quality degradation. SOTA methods in this area reports less than 5% quality drop at 50% unstructured sparsity [32]. However, its irregular memory access complicates hardware acceleration [33] and performance often matches or falls bellow dense counterparts due to overheads in parameter managements [34].

Structured pruning targets entire model components (e.g. channels, layers, Transformer blocks), making it hardware-friendly [34], but causing significant quality degradation due to removal of large parameter groups [35]. SOTA methods in this area provide up to 13% quality degradation at 50% structured weight sparsity [36].

Semi-structured pruning removes N out of every M consecutive weights and thus

offers a favorable compromise between accuracy and hardware efficiency across various GPU architectures [37]. However, SOTA approaches in this field still provides up to 5% quality degradation under 2:4 sparsification pattern [38].

Moreover, with weight pruning the degradation in model performance extremely increases as the proportion of pruned parameters grows [39]. In addition, pruning process often introduces computational overhead on additional inference on calibration data or further training to recover the LLM performance [40].

1.2.4 Backpropagation elimination

Backpropagation elimination involves replacing the traditional backpropagation method with alternative learning rules that are less computationally expensive. The primary motivation behind this is to achieve significant reductions in both memory and time requirements. This scientific field is represented by two main directions: zeroth-order methods and localized learning strategies.

Zeroth-order methods estimate gradients using only two forward passes. A notable advancement in this category is the Memory-efficient Zeroth-order Optimizer (MeZO) [23], that efficiently implements the ZO-SGD method [41]. It has been shown to deliver performance on par with traditional backpropagation fine-tuning across various tasks. Notably, MeZO achieves up to a twelvefold reduction in memory use and up to a twofold decrease in training time. However, it requires considerably more iterations than conventional fine-tuning and only reaches acceptable levels of accuracy when combined with a specifically tailored prompt [23].

The second category involves localized learning rules, particularly the Hebbian rule. The Hebbian rule [42] draws inspiration from the structural and functional organization of the human brain. It suggests that the strength of synaptic connections between neurons increases when the neurons activate simultaneously and weakens when they do not. An efficient combination of localized learning rule with backpropagation provides an acceleration by up to 77%, however introduces a substantial quality

degradation of 3.04% on average [43].

1.2.5 Parameter-Efficient Fine-Tuning

PEFT emerges as a leading method for accelerating the fine-tuning of LLMs. PEFT focuses on updating a minimal subset of the model’s parameters, significantly reducing time and memory costs, but preserving or even enhancing model accuracy [29]. In the context of PEFT, $\Delta\Phi$ is encoded by a much smaller-sized set of parameters Θ , so that $|\Theta| \ll |\Phi_0|$, $\Delta\Phi = \Delta\Phi(\Theta)$. Thus, PEFT aims at maximizing the fine-tuning objective over Θ (4).

$$\max_{\Theta} \sum_{(x,y) \in D} \sum_{t=1}^{|y|} \log(\mathcal{M}_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})) \quad (4)$$

LoRA [19] is de-facto standard approach in the industry, that freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly decreasing the number of trainable parameters for downstream tasks. Let $W_0 \in \mathbb{R}^{d_1 \times d_2}$ denote pre-trained weights of an arbitrary layer, x is an input data. Then, inference of this layer, denoted as h can be expressed as $h = W_0x$. Applying *LoRA* to that layer modifies its forward pass as follows (5).

$$h = W_0x + \Delta W_x = W_0x + BAx, \quad (5)$$

where $W_0, \Delta W \in \mathbb{R}^{d_1 \times d_2}$, $A \in \mathbb{R}^{r \times d_2}$, $B \in \mathbb{R}^{d_1 \times r}$, $r \ll (d_1, d_2)$. A, B matrices are called *LoRA* adapters.

By greatly decreasing the number of model parameters, *LoRA* provides up to 50% memory reduction and 150% acceleration compared to full fine-tuning. Although *LoRA* is a state-of-the-art approach, it still has several drawbacks. First, *LoRA* lacks the nuanced capability for more subtle weight adjustment during the fine-tuning [44]. Second, *LoRA* adapters’ initialization approach could lead to a slower model convergence [45]. Third, *LoRA* overlooks the varying importance of different weight ma-

trices [46]. Finally, *LoRA* falls short of matching the performance achieved by full fine-tuning, as it applies uniform updates across all LLM layers and does not induce more substantial modifications in the most critical layers of the LLM [47].

DoRA [44], PiSSA [45], and AdaLoRA [46] are deemed promising improvements of *LoRA*.

DoRA [44] approach suggests decomposing the pre-trained weights W_0 into *magnitude* and *direction* components (6). *Magnitude* is a trainable vector $M = \|W_0\|_c$, which represents a vector-wise norm of a matrix across each column. *Direction* is the ratio of the pre-trained weight matrix to its norm. The *direction* component is updated using *LoRA* [19].

$$W = M \frac{W_0 + BA}{\|W_0 + BA\|_c} \quad (6)$$

DoRA provides up to 3% of quality improvement on the fine-tuning of LLaMA2-7B and LLaMA2-13B compared with *LoRA*.

AdaLoRA [46] approach adaptively allocates the parameter budget among the weight matrices according to their importance score. It re-parametrizes the incremental updates in the form of Singular Value Decomposition and modifies the *LoRA* forward pass as follows (7).

$$h = W_0x + \Delta W_x = W_0x + P\Lambda Q, \quad (7)$$

where $P \in \mathbb{R}^{d_1 \times r}$ and $Q \in \mathbb{R}^{r \times d_2}$ represent singular vectors of ΔW , the diagonal matrix $\Lambda \in \mathbb{R}^{r \times r}$ contains the singular values $\{\lambda_i\}_{1 \leq i \leq r}$. *AdaLoRA* provides 1% of quality improvement on DeBERTa-V3-base fine-tuning compared with *LoRA*. However, both *DoRA* and *AdaLoRA* still incurs uniform updates of all LLM layers with a little attention to the critical initial projections[47].

PiSSA [45] approach modifies the *LoRA* by initializing A and B adapter matrices with the principal components of the original matrix W computed utilizing Singular

Value Decomposition (8).

$$\begin{aligned}
W &= USV^T, \\
W^{res} &= U_{[:,r:]} S_{[r:,r:]} V_{[:,r:]}^T, \\
A &= U_{[:,r:]} S_{[r:,r:]}^{\frac{1}{2}}, \\
B &= S_{[r:,r:]}^{\frac{1}{2}} V_{[:,r:]}^T,
\end{aligned} \tag{8}$$

where S is a diagonal matrix, which represents the singular values arranged in descending order. Thus, trainable adapters contain the most essential directions of W , that is why training adapters mirror the process of the entire model fine-tuning. Moreover, during the initial fine-tuning steps *PiSSA* significantly update an essential first layers of LLM, evoking the trajectory of full fine-tuning [47] and provides up to 21.05% of quality improvement on LLaMA2-7B fine-tuning compared with *LoRA*.

1.2.6 Summary

Therefore, there exist various approaches for fine-tuning acceleration.

- **Data reduction** techniques provide significant acceleration by reducing the size of the training dataset, lowering the computational resources required for model training. However, these methods introduce extra overhead on importance estimation of samples and lead to quality degradation due to removal of training samples from the dataset.
- **Quantization** strategies significantly reduce both the time and memory required for fine-tuning, but generally result in a noticeable decline in the model quality.
- **Pruning** techniques provide substantial acceleration and memory reduction, but lead to a quality drop, bring time overhead on additional inference on calibration data or retraining, and demand special hardware for efficient computation.

- **Backpropagation elimination** methods significantly reduce computational demands by lowering memory and training time, but often require more iterations to achieve satisfactory accuracy or introduce quality degradation compared to backpropagation-only approaches.

Hence, Parameter-Efficient Fine-Tuning is considered the most prominent approach, since its state-of-the-art *LoRA* method provides noticeable acceleration of up to 150% with memory reduction of 50% while maintaining the quality comparable to full fine-tuning. However, significant optimization opportunities of *LoRA* are presented by selectively attaching adapters to a subset of the important Transformer blocks.

1.3 Existing approaches for assessing the importance within Large Language Model

The evaluation of importance can be implemented for various structures, ranging from individual weights to Transformer blocks, and even entire model architectures. The overview of observed methods is presented in Table 1.

1.3.1 Importance of separate weights

The Lottery Ticket Hypothesis [48] suggests that within a randomly-initialized, dense model exists a smaller subnetwork, or the *winning ticket* which can achieve the same level of test accuracy as the full model if trained independently, often within the same or even fewer training iterations. This phenomenon has been observed in a range of architectures within both the Computer Vision and Natural Language Processing fields. For instance, a subnetwork within the LeNet architecture, constituting only 51.3% of the original network’s weights, has demonstrated the ability to surpass the full model in terms of test accuracy more rapidly. Similarly, the *winning ticket* for the BERT-base model, albeit with a sparsity ratio of 90%, can perform within one standard deviation of the full model’s performance [49].

Table 1: Summary of considered approaches for importance estimation. Negative value is a quality drop.

Structure	Reference	Best quality, (↑)	Worst quality, (↑)	Acceleration, (↑)
Weight	[48]	1.7	–	N/A
	[49]	0.8	–	N/A
	[8]	-2.4	–	N/A
	[50]	-15.4	–	N/A
	[17]	-5.5	–	N/A
	[12]	-2.9	–	N/A
	[14]	2.2	–	N/A
	[51]	1.1	–	N/A
Layer	[52]	-19.5	–	N/A
	[53]	-2.4	–	N/A
	[54]	5.3	–	N/A
	[55]	4.1	–	N/A
	[56]	-5.7	–	N/A
	[57]	N/A	–	N/A
Transformer block	[58]	18.5	–	-14.1
	[59]	12.3	–	3.4
	[15]	N/A		-6.7
	[16]	27.6		-8.5
	[60]	23.3		-7.4
Model	[61]	N/A		4.2
	[9]	N/A		1.5

Developing the idea of the importance of individual weights within the model, the concept of systematic outliers in LLMs was introduced, which are defined as values that deviate significantly from the average of their distribution [62]. These outliers influence the model to predominantly predict the most frequent tokens, thereby affecting the overall quality. Notably, the emergence of these outliers occurs in the Multi-Head Attention module of the Transformer block, significantly influenced by the SoftMax operation with dominant attention weights increasing exponentially for key values (9), while other weights approach zero (10). This imbalance in attention weights results in significant updates of the projection weights K and V during back-propagation. If the MHA module produces extreme values due to attention outliers, as the output from the MHA module passes through the MLP block, these anomalies

are further amplified by the non-linear activation inside the MLP module and concentrated in the down-projection matrix. Thus, the combination of steep gradients and non-linear transformations fosters the emergence of outliers in both activations and weights.

$$A_{xj} = \frac{\exp(M)}{\exp(M) + (n - 1)} \approx 1 \quad (9)$$

$$A_{xk} = \frac{1}{\exp(M) + (n - 1)} \approx 0, \quad (10)$$

let $M = \frac{W_q(W_k^T)_j}{\sqrt{d_h}}, \frac{W_q(W_k^T)_k}{\sqrt{d_h}} = 0$, then $\frac{W_q(W_k^T)_j}{\sqrt{d_h}} \gg \frac{W_q(W_k^T)_k}{\sqrt{d_h}}$, n is the number of elements.

Various types of outliers were suggested [62], such as:

1. **Activation outliers inside layer.** For layer outputs $h_l \in \mathbb{R}^{B \times H}$ with B being the batch size, H being the hidden dimension, and $\mu_h = \frac{1}{B \cdot H} \sum_{i,j} |h_{i,j}|$ being the mean absolute value of layer outputs h_l , the set of activation outliers inside the Transformer block is defined as follows (11).

$$O = \{(i, j) \mid |h_{i,j}| > \tau \cdot \mu_h\}, \quad (11)$$

where τ is a predefined threshold.

2. **Activation outliers inside the down-projection weight.** For down projection in-puts $x_l^{down} \in \mathbb{R}^{B \times H}$ and $\mu_{x^{down}} = \frac{1}{B \cdot H} \sum_{i,j} |x_{i,j}^{down}|$, the set of activation outliers is defined as follows (12).

$$O = \{(i, j) \mid |x_{i,j}^{down}| > \tau \cdot \mu_{x^{down}}\} \quad (12)$$

3. **Weight outliers.** For projection weights $W \in \mathbb{R}^{O \times I}$ with O being output dimension, I being input dimension, and $\mu_{w_i} = \frac{1}{I} \sum_j |w_{i,j}|$ being the row-wise mean absolute value of W , the set of weight outliers is defined as follows (13).

$$O = \{(i, j) \mid |w_{i,j}| > \tau \cdot \mu_{w_i}\} \quad (13)$$

4. **Attention outliers.** For cumulative attention scores $A \in \mathbb{R}^{L \times L}$ with L being a sequence length, $\hat{A}_j = \sum_{i=1}^L A_{i,j}$ being the cumulative attention contribution for token j , and $\mu_A = \frac{1}{L} \sum_j \hat{A}_j$, the set of attention outliers is defined as follows (14).

$$O = \{j \mid \hat{A}_j > \tau \cdot \mu_A\} \quad (14)$$

These systematic outliers significantly impact model quality and pose challenges for model compression techniques, such as pruning and quantization [62].

In terms of pruning, the removal of weight outliers can have a devastating effect on model performance, leading to a quality degradation of up to 50% for LLaMA2-7B in few-shot learning tasks [8]. Conversely, preserving these crucial weights while eliminating the less important ones results in a quality degradation, not exceeding 2% [8].

However, even preserving these critical weights can still lead to quality drop. The Magnitude pruning approach [63] (15), which removes smallest absolute values weights, sometimes results in suboptimal pruning and a degradation in a model performance on 15% for the LLaMA2-7B model [17].

$$i_\xi = |W|, \quad (15)$$

where $|W|$ is the absolute value of weight matrix.

Even more advanced methodologies such as Wanda [17] (16), which enhances sparsity by considering both weight magnitudes and input activations are still facing a similar problem. Despite demonstrating a 10% perplexity improvement for the LLaMA2-7B model compared to the traditional magnitude pruning, Wanda still leads to 5-15% of quality drop compared with the original dense model.

$$i_T = |W| \cdot \|X_T\|_2, \quad (16)$$

In terms of quantization, several strategies have been developed to account for the significance of individual weights. Outlier-aware quantization, for instance, has

demonstrated superior performance, exceeding the state-of-the-art SmoothQuant [64] method by 1% on both the LLaMA2-7B and LLaMA2-13B models [8]. Moreover, skipping the quantization of these extremely important weights and maintaining them in the FP16 data format can enhance model quality by as much as 67% compared to their full quantization [12]. Additionally, sensitivity-based non-uniform quantization, which optimizes bit precision assignments and stores outliers and sensitive weight values in a space-efficient sparse format, results in less than 2% quality degradation when transitioning from a 16-bit to a 4-bit format [65].

1.3.2 Importance of layers

Moving forward, the significance of various layers within the Transformer model has been extensively studied, revealing intricate details about their functionalities and relative importance.

Previously, experiments on re-initialization and re-randomization robustness of Transformer layers inside LLMs were conducted [52]. This implies re-initializing or assigning random parameters for different model layers, respectively, while keeping the rest of the model parameters intact. As a result, it was revealed that layers within these models can be classified as either robust or critical: robust layers, when reset to their initial values, do not significantly impact the model’s performance. This suggests that not all layers are equally important, and some, especially in over-parametrized models trained with stochastic gradient descent, are inherently less critical due to the model self-restricting complexity [52].

Besides, it was observed that individual Attention heads within the Transformer model serve distinct roles in the generation process [53]. Specifically, certain Attention heads are pivotal for identifying syntactic and dependency relationships, highlighting the specialized functions of these components within the architecture. This research also demonstrated that the encoder retains its efficacy despite a considerable reduction in the number of Attention heads, indicating a potential for optimization in

the model design [53].

In a related vein, it was noted that perplexity – a measure of model performance – is more closely associated with overall model capacity than merely the presence of self-attention mechanisms [54]. This finding suggests that increasing the complexity of the model could potentially be more beneficial than focusing solely on the inclusion of self-attention layers.

Further expanding on the theme of layer importance, Tyukin et al. [55] reported that reducing the number of MHA layers by 40% leads to only a minimal decrease in performance. In contrast, a similar reduction in MLP layers results in a significantly greater decline in output quality of up to 60% for the LLaMA2-7B model. This disparity underscores the critical role of MLP layers in maintaining the overall effectiveness of the Transformer model.

Adding to the discourse, He et al. [56] discovered a high level of redundancy within MHA layers, suggesting that a substantial portion of these layers could be removed without notably impairing the model’s performance. This finding points towards potential efficiencies in model architecture by minimizing unnecessary layers.

Lastly, Tian et al. [57] elucidated the intricate relationship between attention logits and MLP weights, proposing that explicit updates to self-attention may be redundant. This is due to the implicit incorporation of self-attention functionalities in the lower MLP layers, further emphasizing the possibility of streamlining Transformer architectures without sacrificing performance.

1.3.3 Importance of Transformer blocks

Moving on to larger structures within LLMs, the saliency of the whole Transformer blocks could be estimated.

Various methodologies have been proposed to assess the importance of Transformer blocks in LLMs. Inspired by the aforementioned Wanda [17] approach, OWS [59] selectively prunes Transformer blocks based on their outlier distribution, stating

that outliers mostly presented in the first and the last Transformer blocks. *OWS* allows to achieve a 12% reduction in memory usage and a modest 3% quality improvement over *LoRA*. However, *OWS*'s extensive parameters updates make it time-inefficient.

UIDL [15] states that layer-to-layer evolution of representations for a Transformer is given by a residual iteration equation (17).

$$x^{(l+1)} = x^{(l)} + f(x^{(l)}, \theta^{(l)}), \quad (17)$$

where $x^{(l)}$ is the multi-dimensional input, $\theta^{(l)}$ is parameters vector for layer l , $f(x, \theta)$ describes the transformation of one Transformer block.

To successfully prune the n layers before layer l it would be better if the input to the pruned block will be very similar to the output of the pruned block (18).

$$x^{(l)} = x^{(l-n)} + \varepsilon, \quad (18)$$

where ε is a constant indicating the degree to which one input may differ from another.

Therefore, the least important Transformer blocks could be defined as the least affecting on the model output and identified with cosine similarity between input and output to the Transformer block (19).

$$i_T = \operatorname{argmin} \frac{1}{\pi} \arccos\left(\frac{X_T \cdot Y_T}{\|X_T\|_2 \cdot \|Y_T\|_2}\right), \quad (19)$$

where X_T and Y_T are the input and output to the Transformer block, respectively, $\|\cdot\|_2$ is a Frobenius norm.

The proposed approach allows to achieve a 40% reduction in layers without a quality drop for the LLaMA2-70B model [15]. However, *UIDL* requires additional inference on calibration data, which increases time overhead. Besides, cosine similarity was efficiently utilized in ShortGPT [58] leading to 18% of acceleration with quality drop of 14%.

Another approach is *SLEB* [16], which utilizes perplexity as a metric to evaluate Transformer block importance based on the assumption, that redundant blocks contribute less to the model's output, and their removal leads to smaller degradation in a

perplexity (20).

$$i_T = \exp\left\{-\frac{1}{S} \sum \log p_{\theta}^n(X_T^{(S)} | X_{<T}^{(S)})\right\}, \quad (20)$$

where X_T is the input to Transformer block, S is a sequence length.

SLEB allows achieving 27% of acceleration with 8% of quality drop on LLaMA2-7B compared to the original model [16]. This method also requires extra inference on calibration data, leading to higher time costs. In addition, utilizing perplexity as an importance metric is unsuitable for scenarios that require dynamic freezing of non-essential Transformer blocks, as they inhibit gradient calculations.

Shortened-LLaMA [60] measures the significance of Transformer blocks by their impact on loss, assessing the error caused by the removal of a weight parameter (21).

$$i_T = L(W_T + \Delta W_T) - L(W_T) = \nabla_W L \Delta W_T, \quad (21)$$

where W_T is a Transformer block weight, L is a loss.

Shortened-LLaMA allows achieving 23% of acceleration with 7% of quality drop on LLaMA2-7B compared to the original model [60]. Notably, like *SLEB*, *Shortened-LLaMA* incurs additional time overhead due to the need for inference on the calibration dataset. In addition, loss-based metric is also inapplicable for dynamic removal of Transformer blocks.

1.3.4 Neural Architecture Search

Finally, the importance of the full architectures could be estimated. Such approaches have been developed within the field of Neural Architecture Search. Neural Architecture Search (NAS) is a technique for automating the design of models. Instead of manually developing neural network architectures, NAS algorithms explore and discover optimal architectures for specific tasks.

For this purpose NAS utilizes a special controller to estimate the performance of a predicted neural architecture on a proxy dataset. However, this evaluation method, particularly when based on validation accuracy, can be computationally prohibitive.

For example, NASnet [66] approach requires more than 2000 GPU days to identify optimal model architecture.

In response to the high computational demands of traditional NAS methods, zero-cost proxies present a more resource-efficient alternative. Zero-cost methods rely heavily on importance heuristics to approximate the performance of neural architectures without the need for extensive training. For example, the TF-TAS [61] approach utilizes synaptic diversity for the MHA module and synaptic saliency for the FFNN module. TF-TAS improves upon existing techniques, leading to a 4% quality improvement compared with manually designed ResNet and MobileNet-V3. At the same time, TF-TAS significantly reduces the number of GPU days required to estimate the predicted architecture compared with AutoFormer [67] and ViTAS [68].

A comprehensive comparison study of 15 zero-cost proxies applicable to Transformer architectures and LLMs [9] revealed varying levels of its effectiveness. These proxies were assessed through Kendall and Spearman correlations, with values ranging from -1 to 1, where higher values indicate better predictive accuracy of neural architecture rankings vis-à-vis fully trained models. Despite their utility, many proxies exhibited low correlation values, with Kendall's ranging from 0.01 to 0.5 and Spearman's from 0.1 to 0.5 [9].

1.3.5 Summary

Therefore, various structures of LLMs, including individual weights, layers, and Transformer blocks, as well as entire model architectures, can be evaluated in terms of their importance, specifically regarding their impact on the overall performance of the model. Such importance assessment methods are actively utilized in different optimization strategies for LLM deployment during inference, such as sparsification and quantization. Among the structural components considered, the Transformer block is particularly noteworthy. Existing research has already demonstrated the potential for compressing LLM, achieving up to a 30% time and memory usage reduction by

assessing the importance of Transformer blocks and retaining only the most significant ones. In contrast, approaches that focus on individual elements or layers have not achieved comparable resource savings, while methods focusing on entire architectures require substantial time and memory resources and utilized only within a Neural Architecture Search field. Hence, the analysis of Transformer block importance emerges as the most promising direction in this context.

However, while existing solutions for estimating the Transformer block importance successfully minimize computational costs, particularly in the field of model inference, they often lead to a substantial quality degradation of an LLM performance. As a step towards addressing this challenge, the assessment of the importance of Transformer blocks can be further utilized within a fine-tuning field improving model quality, concurrently optimizing fine-tuning costs.

The next chapter presents a novel fine-tuning strategy, that dynamically allocates *LoRA* adapters to essential Transformer blocks based on efficient importance analysis to further accelerate the fine-tuning process, while preserving the model quality.

Chapter 2. Development of an approach to accelerate the fine-tuning through importance analysis

The discussed idea of estimating the importance of Transformer blocks formed the basis of the proposed method *ID-LoRA (Importance-driven LoRA)*. *ID-LoRA* selectively assigns and re-assigns *LoRA* adapters based on Transformer blocks importance during the fine-tuning process with subsequent reduction of the number of Transformer blocks.

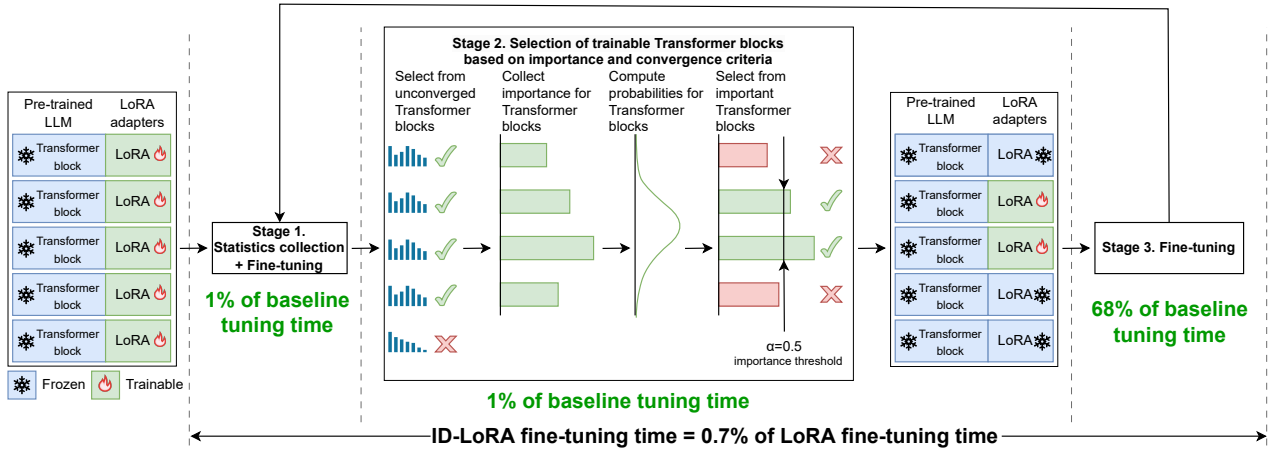


Figure 1: Key idea of proposed fine-tuning method: *Importance-driven LoRA*.

Hyperparameters: let h denote history defined as the number of steps to collect importance statistics, α denote the ratio of Transformer blocks to be selected for fine-tuning.

Let us suppose that we are faced with the task of fine-tuning a pre-trained LLM ($\mathcal{M}_\Phi$) on a specific dataset (D). Initially, the fine-tuning process is divided into several periods, during which the algorithm switches the trainable Transformer blocks. Each *ID-LoRA* period has 3 phases: statistics collection, selection of trainable Transformer blocks, and fine-tuning of the selected blocks. At the beginning of each stage, layer-wise importance statistic is collected during h steps. After that, importance sampling of Transformer blocks is applied, and the ratio of the most important Transformer blocks (α) is selected for further fine-tuning. At the same time, *LoRA* A, B adapters for not-trainable Transformer blocks are merged up to the end of the fine-

tuning.

As an alternative or a complementary strategy to importance analysis, a convergence analysis could be applied to further reduce the number of trainable Transformer blocks. The convergence analysis was not proposed by the author, therefore the detailed explanation of this part of the method can be find in the article [69].

The full *ID-LoRA* method is described in Algorithm 1.

Algorithm 1 *ID-LoRA* (α, h)

```

1:  $D$ : fine-tuning dataset
2:  $\mathcal{M}_\Phi$ : pre-trained model
3:  $\alpha$ : trainable blocks selection ratio
4:  $h$ : number of accumulation steps
5:
6:  $\mathcal{Q} \leftarrow \{\mathcal{M}_\Phi\}$ : trainable Transformer blocks
7:  $I \leftarrow \{\}$ : accumulated importance scores
8:  $\mathcal{W} \leftarrow \{\}$ : accumulated layer weights
9: for epoch  $i = 1, \dots, N$  do
10:   for train step  $i = 1, \dots, K$ , global batch  $b \in D$  do
11:      $inp, labels \leftarrow b$ : data for forward
12:     if  $i < h$  then
13:       {Statistics Collection + Fine-tuning Phase}
14:       for layer  $\xi$  in  $\mathcal{M}_\Phi$  do
15:          $inp, out \leftarrow forward(\xi, inp)$ 
16:          $I \leftarrow I \cup i(\xi, inp, out)$ : calculate layer importance
17:          $grads \leftarrow backward(loss(out, labels))$ 
18:          $apply\_gradients(grads)$ 
19:       end for
20:     else if  $i = h$  then
21:       {Selection Phase}
22:       for  $\mathcal{T}$  in  $\mathcal{Q}$  do
23:          $I_{\mathcal{T}} \leftarrow I(\{w_j : w_j \in \mathcal{T}\})$ : aggregate importance of Transformer blocks (24)
24:       end for
25:        $P \leftarrow softmax(I_{\mathcal{T}})$ : select with (22)
26:        $T \leftarrow \alpha * top\_p(\mathcal{Q}, P)$ : (23)
27:        $Unfreeze(R)$ 
28:        $Freeze(\mathcal{Q} \setminus R)$ 
29:        $\mathcal{Q} \leftarrow R$ 
30:     else
31:       {Fine-tuning Phase}
32:        $inp, out \leftarrow forward(inp)$ 
33:        $grads \leftarrow backward(loss(out, labels))$ 
34:        $apply\_gradients(grads)$ 
35:     end if
36:   end for
37: end for

```

2.1 Importance-based Transformer blocks selection

2.1.1 Statistics collection phase

The first phase of *ID-LoRA* is dedicated to statistics collection over several iterations of fine-tuning.

Step 1: Importance metric selection According to the results of the first year’s Course work [70], cosine similarity importance metric, which is employed in *UIDL*, is the preferred option as this approach demonstrated robust performance after Transformer blocks pruning across different LLM scales and a broad range of tasks. Moreover, the cosine similarity focuses on the direction of vectors rather than their magnitude, which allows to preserve Transformer blocks that contain essential semantic information for LLM performance [71]. Therefore, the cosine similarity was applied as an importance estimation method in the proposed approach.

Step 2: Statistics collection During h steps of fine-tuning, importance scores are accumulated for each layer of the model. The cosine similarity importance score is calculated between input and output of each Transformer blocks within an LLM.

2.1.2 Selection phase

Step 3: Importance scores aggregation Upon reaching the h number of steps, the importance scores of each Transformer block are averaged.

Step 4: Probability calculation Based on the importance scores, selection probability for each Transformer block is computed (22).

$$P(T) = \frac{e^{I(T)}}{\sum_{k=1}^{\mathcal{M}_{\Phi}} e^{I(k)}}, \quad (22)$$

where I denotes Transformer block importance score.

Step 5: Top-P sampling Next, the Top-P sampling (23) is applied, which samples Transformer blocks with the highest probability scores until the sum of the scores reaches the specified threshold value.

$$Top_P = \{T_k \in \mathcal{M}_\Phi : \sum_{k=1}^{\mathcal{M}_\Phi} P(T_k) \leq p\}, \quad (23)$$

where p denotes the threshold.

Step 6: Transformer blocks selection Then, the sampling probability for each Transformer block is recalculated (22). Finally, α of the most important Transformer blocks are selected for further fine-tuning. Concurrently, for non-trainable Transformer blocks *LoRA* A, B adapters are merged up to the end of the fine-tuning. Merging non-updatable adapters allows to further accelerate the fine-tuning process and increase memory reduction. Obviously, with each training period, the number of trainable Transformer blocks diminishes. As a result, fine-tuning stops once there are no more trainable Transformer blocks.

After the fine-tuning phase *LoRA* A, B adapters are merged inside the original LLM for the following deployment.

2.2 Key differences from existing approaches

- Existing approaches utilize importance-based pruning of Transformer blocks on inference, introducing considerable quality degradation. In contrast, *ID-LoRA* integrates the pruning process into the Parameter-Efficient Fine-Tuning with *LoRA*, thereby preventing performance degradation.
- Prior methods incur extra computational costs by requiring an additional inference on calibration data for importance estimation. Conversely, *ID-LoRA* identifies essential Transformer blocks during the fine-tuning process without additional overhead and calibration data.

- Current state-of-the-art approaches manually attach the *LoRA* adapters to all Transformer blocks before the fine-tuning process. On the contrary, *ID-LoRA* offers an innovative approach that enables automatic assignment of *LoRA* A , B adapters only to important Transformer blocks.

Chapter 3. Evaluation of the proposed method

To assess the feasibility of the *ID-LoRA*, comprehensive evaluation of the proposed solution has been applied with detailed results being presented in the article [69].

3.1 Software implementation

The software implementation is made in a form of Python package, adhering to the principles of object-oriented programming, and is fully compatible with all the considered LLMs. The code is the intellectual property of Huawei and cannot be disclosed in this work.

3.2 Protocol

For the evaluation purposes, six LLMs of different scale were used: GPT2-M, LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B. All LLMs except GPT2-M were taken from HuggingFace hub [72], [73], [74], [75], [76]. The GPT2-M model was taken from *transformers v4.33.2*.

Table 2: Hardware and software setup used for the training and evaluation of the *ID-LoRA*.

	GPT2-M	LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B
CPU	CPU 32 cores with 3.0GHz	CPU 32 cores with 2.90GHz
OS	Ubuntu 20.04.6 LTS	Ubuntu 20.04.04 LTS
Driver Version	515.43.04	535.183.01
Kernel Version	5.4.0-176-generic	5.15.0-125-generic
CUDA Toolkit Version	11.7.64	12.2
Training Libraries	<i>transformers</i> 4.33.0, <i>PyTorch</i> 2.1.0, <i>DeepSpeed</i> 0.15.4, <i>peft</i> v.0.18.0	<i>transformers</i> 4.45.0, <i>PyTorch</i> 2.1.0, <i>DeepSpeed</i> 0.15.4, <i>peft</i> v.0.18.0

Table 3: Fine-tuning protocol used for the training of *ID-LoRA*.

	GPT2-M	LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B
Sequence Length	512	256
Learning Rate	6e-4	1e-5
AdamW β_1	0.9	0.9
AdamW β_2	0.99	0.99
Weight Decay	0.01	0
Epochs	2	2
Precision	fp16	bf16
<i>LoRA</i> r	16	16
<i>LoRA</i> α	32	32

As *LoRA* is the state-of-the-art method for accelerating fine-tuning, it was utilized as the primary baseline in computational experiments evaluating both performance and speed. Alongside *LoRA*, several cutting-edge modification — including *DoRA* [44], *PiSSA* [45], and *AdaLoRA* [46] – were incorporated as comparison benchmarks during assessment. The implementations for these methods were sourced from the *peft* v.0.18.0 library [77].

Details regarding the hardware and software used for training can be found in Table 2, while the specific training protocols for each model are outlined in Table 3.

After the fine-tuning, LLMs were evaluated on a wide range of commonsense reasoning datasets, including PIQA [78], HellaSwag [79], Winogrande [80], ARC [81], OpenbookQA [82], MMLU [83] to estimate zero-shot accuracy using the *lm-evaluation-harness* package [84]. In addition, zero-shot PPL was measured on language modelling dataset WikiText2 [85]. In this work, an average accuracy and perplexity are presented. The detailed results for each of these datasets are presented in the paper [69].

Table 4: Quality and acceleration results of fine-tuning with *ID-LoRA*, baseline *LoRA* and reference *DoRA*, *PiSSA*, and *AdaLoRA* approaches.

Model	Method	Average accuracy, ($\uparrow$)	Perplexity, ($\downarrow$)	Acceleration, (%, $\uparrow$)
GPT2-M	<i>LoRA</i> (baseline)	37.5	173.1	–
	<i>DoRA</i>	37.0	184.2	-21.6
	<i>PiSSA</i>	34.9	172.1	0.9
	<i>AdaLoRA</i>	36.9	179.2	-1.3
	<i>ID-LoRA</i>	38.0	72.7	25.2
LLaMA2-7B	<i>LoRA</i> (baseline)	59.9	9.2	–
	<i>DoRA</i>	59.8	9.2	-407.3
	<i>PiSSA</i>	60.9	9.4	1.7
	<i>AdaLoRA</i>	59.3	9.2	-51.6
	<i>ID-LoRA</i>	59.4	9.2	29.2
LLaMA2-13B	<i>LoRA</i> (baseline)	63.8	7.7	–
	<i>DoRA</i>	63.7	7.7	-203.3
	<i>PiSSA</i>	64.5	7.9	-0.3
	<i>AdaLoRA</i>	63.1	7.7	-92.9
	<i>ID-LoRA</i>	63.6	7.7	44.8
Qwen2.5-3B	<i>LoRA</i> (baseline)	62.9	15.5	–
	<i>DoRA</i>	62.9	15.3	-201.0
	<i>PiSSA</i>	63.5	11.8	-0.6
	<i>AdaLoRA</i>	62.3	15.1	-71.5
	<i>ID-LoRA</i>	62.9	15.5	28.6
Qwen2.5-7B	<i>LoRA</i> (baseline)	66.6	8.8	–
	<i>DoRA</i>	66.6	8.8	-761.3
	<i>PiSSA</i>	66.7	9.0	0.7
	<i>AdaLoRA</i>	66.4	8.7	-42.6
	<i>ID-LoRA</i>	66.5	8.7	33.2
Qwen2.5-32B	<i>LoRA</i> (baseline)	70.0	6.0	–
	<i>PiSSA</i>	70.8	6.1	-1.0
	<i>AdaLoRA</i>	69.6	6.0	-92.5
	<i>ID-LoRA</i>	69.6	5.6	37.5

3.3 Analysis of the obtained results

Table 4 shows the results of computational experiments conducted using the selected models and methods. The performance of the *DoRA* approach with the Qwen2.5-32B model is not reported because its memory demands exceeded the capabilities of the available hardware.

For GPT2-M model the hyperparameter α of *ID-LoRA* is 0.8, for all other models $\alpha=0.6$. The GPT2-M model necessitates updating a larger number of layers compared to the other models, since it has much fewer trainable parameters compared to other

considered LLMs. In contrast, the greater depth of LLM means that updating a smaller proportion of its layers still involves a substantial absolute number being fine-tuned. For all evaluated models the hyperparameter $h = 10$.

ID-LoRA provides an average acceleration of 33.1% with a considerable maximum speedup of 44.8% for LLaMA2-13B with a quality decrease of less than 0.2%. In addition, *ID-LoRA* significantly outperforms *PiSSA*, *DoRA*, and *AdaLoRA* from the point of acceleration, and provides 0.4% of quality improvement compared to *AdaLoRA* and comparable performance to *DoRA*. However, it leads 0.2% quality decrease compared to *PiSSA* as *PiSSA* employs an SVD initialization of *LoRA* A , B adapters, which provides them with the most important information from the original weights W .

3.4 Sensitivity analysis

Sensitivity analysis of α and h hyperparameters of *ID-LoRA* and importance metrics is presented in Figure 2, Figure 3, and Figure 4. They were analyzed in terms of their impact on the resulting quality and acceleration.

3.4.1 Analysis of hyperparameter α

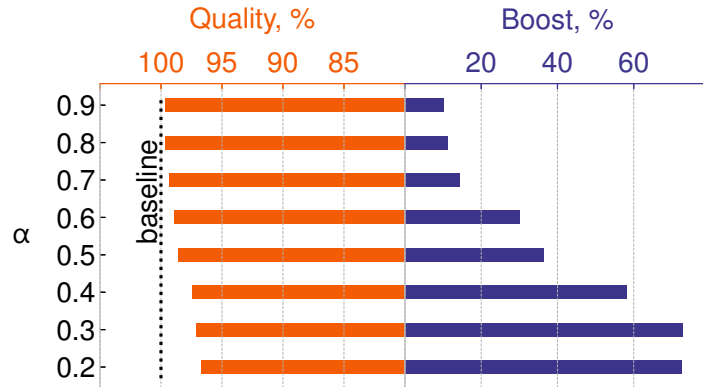


Figure 2: Sensitivity analysis of *ID-LoRA* to the α hyperparameter. h is 10, importance metric is cosine similarity.

Ablation study of the α hyperparameter of *ID-LoRA* is presented in Figure 2. Obtained results demonstrate that increasing α significantly affects both acceleration and model quality. In particular, a maximum speedup of 72.6% is observed at $\alpha = 0.2$, where only 20% of the Transformer blocks are trained. However, it leads to a 3.3% increase in perplexity, suggesting underfitting. Minimal quality degradation (0.3%) occurs at higher α values (0.8 and 0.9), where 80-90% of the Transformer blocks are trained initially. $\alpha = 0.6$ is identified as the optimal setting for balancing speed and quality.

3.4.2 Analysis of hyperparameter h

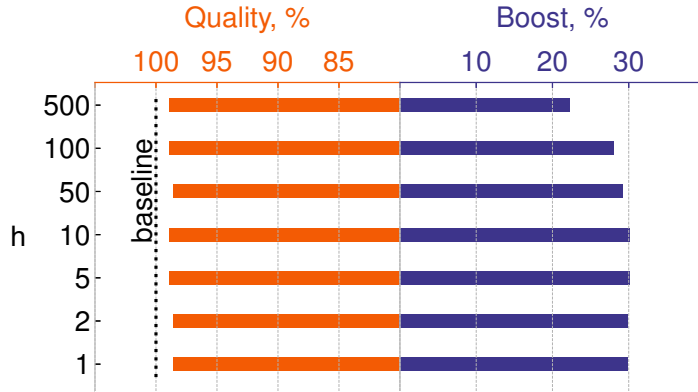


Figure 3: Sensitivity analysis of *ID-LoRA* to the h hyperparameter. α is 0.6, importance metric is cosine similarity.

Ablation study of the h hyperparameter of *ID-LoRA* is presented in Figure 3. According to the achieved results, variations in hyperparameter h do not significantly impact quality, with only a 0.3% change in perplexity was observed between $h = 1$ and $h = 500$. The smallest h value accelerates fine-tuning by 29.8%, indicating that even a single training step can adequately capture the importance of a Transformer block. This observation is supported by the relative stability of model quality across different h settings, as shown in Figure 3. $h = 10$ is identified as the optimal setting for balancing speed and quality.

3.4.3 Analysis of importance metrics

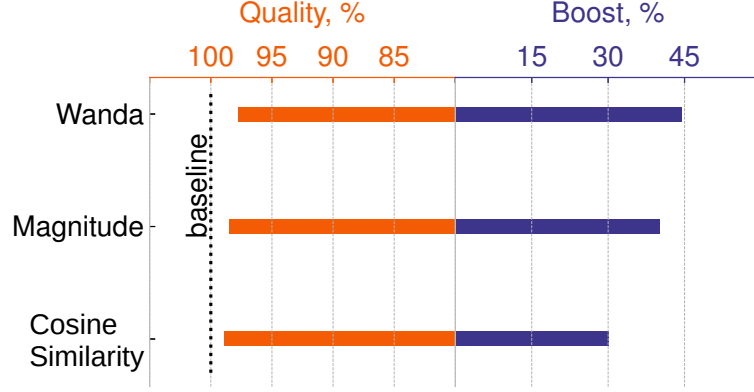


Figure 4: Sensitivity analysis of *ID-LoRA* to considered importance metrics. α is 0.6, h is 10.

An ablation study of importance metrics is presented in Figure 4. Despite the fact, that cosine similarity is considered an optimal choice, an additional ablation study on applying standard pruning metrics, such as Magnitude (15) and Wanda (16) was performed. Notably, Magnitude and Wanda importance metrics are adapted to Transformer block importance estimation as described in Definition 1.

Definition 1. Let T denote Transformer blocks, i_ξ denote importance score for ξ layer inside it, then importance of Transformer block is defined as:

$$I(T) = \frac{\sum_{\xi \in T} i_\xi}{|T|} \quad (24)$$

Sensitivity analysis reveals that the Magnitude metric yields the best balance between acceleration and quality. In contrast, the Wanda metric, despite considering input activations, provides a higher speedup at the cost of increased quality degradation by 2% compared to the baseline. Although cosine similarity decelerates the fine-tuning on 7% compared with the Magnitude, it degrades model quality the least, representing the most suitable choice, as minimizing the drop in model quality is particularly crucial in the context of importance-based fine-tuning. This conclusion is also consistent with the results of last year’s study.

3.5 Summary

In conclusion, the quality and acceleration of the proposed *ID-LoRA* was estimated across different LLM scales (350M, 3B, 7B, 13B, 32B) and a broad range of tasks. According to the obtained results, *ID-LoRA* significantly outperforms the baseline *LoRA* approach and reference *DoRA*, *PiSSA*, and *AdaLoRA* methods from the point of acceleration. *ID-LoRA* provides on average 33% of fine-tuning time speedup compared to *LoRA* approach. Moreover, *ID-LoRA* achieves near-lossless performance with an average quality degradation of 0.2% compared to baseline *LoRA* approach.

According to the conducted ablation study, the hyperparameter $\alpha = 0.6$ and hyperparameter $h = 10$ are considered an optimal choice with importance metric being cosine similarity.

Conclusion

3.1 Results

This work is the result of a two-year research study, which was aimed to address to primary objectives:

- Conduct a comprehensive analysis of current methods used to assess importance of different structures within an LLM, focusing on the evaluation of critical Transformer blocks to enhance the efficiency of the fine-tuning.
- Introduce a novel fine-tuning approach for LLMs, that dynamically assigns *LoRA* adapters during the fine-tuning based on importance analysis of Transformer blocks.

Throughout the research, various modern approaches for reducing time and memory overhead during the fine-tuning were considered with *LoRA* de facto being the state-of-the-art solution in the field. However, *LoRA* can be substantially optimized by estimating the importance of Transformer blocks within the LLM. The concept of "importance" was deemed as the identification of structures that can be removed without significantly affecting model performance. It was found, that existing importance-based compression techniques are mostly utilized during inference, leading to considerable quality degradation. In addition, they provide substantial costs due to additional inference on calibration data.

To address the aforementioned challenges, the following results were obtained:

- The result of the first year's study is a comprehensive overview of existing approaches to assess the importance of Transformer blocks, followed by the conclusion that cosine similarity is the preferred option in terms of quality [70].
- The result of the second year's study is the development of a novel *LoRA*-based fine-tuning approach, *ID-LoRA*, that utilizes importance estimation for efficient reallocation of *LoRA* adapters.

During the evaluation, as a baseline, state-of-the-art *LoRA* approach has been used. In addition, quality and acceleration results of such modifications of *LoRA* as *DoRA*, *PiSSA*, and *AdaLoRA*, were added for reference.

The fine-tuning quality of *ID-LoRA* was estimated on 6 LLMs of different scale, including GPT2-M, LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, and Qwen2.5-32B. After the fine-tuning, zero-shot accuracy was measured on common-sense reasoning datasets and perplexity was measured on WikiText2 (details are provided in Section 3.2).

The results showed that:

- Proposed *ID-LoRA* demonstrates a notable acceleration of 29% for Qwen2.5-3B, 33% for Qwen2.5-7B, 29% for LLaMA2-7B, 45% for LLaMA2-13B, and 38% for Qwen2.5-32B without sacrificing the accuracy compared to baseline *LoRA* approach.
- Proposed *ID-LoRA* provides a considerable speedup compared to baseline *LoRA* and all considered reference fine-tuning acceleration methods.

3.2 Relevance

This research was carried out within a project at Huawei, with the objective of accelerating the fine-tuning of LLMs. The results of this work were presented at the European Conference on Artificial Intelligence 2025 (ECAI, Rank A), Neuroinformatics 2024, and published in the articles in journals indexed in Scopus (Q4, Q3). An individual contribution involves formulating the main concept of the proposed *ID-LoRA* method, which proposes to attach *LoRA* *A*, *B* adapters based on the importance analysis during the fine-tuning process. The second innovation of the method – convergence analysis – was not proposed by an author.

3.3 Future directions

Work in this area is ongoing, with efforts focused on further developing the proposed solution *ID-LoRA* for pre-training and Reinforcement Learning tasks.

References

1. *Tuncer T., Ertam F., Dogan S., Aydemir E., P lawiak P.* Ensemble Residual Networks based Gender and Activity Recognition Method with Signals // The Journal of Supercomputing. — 2020. — Март. — Т. 76. — С. 2119—2138.
2. *Wu G., Suk H.-I.* Deep Learning in Medical Image Analysis // Annual review of biomedical engineering. — 2017. — Март. — Т. 19.
3. *Brown T. B.* [и др.]. Language Models are Few-Shot Learners. — 2020.
4. *Ouyang L.* [и др.]. Training language models to follow instructions with human feedback. — 2022.
5. *DeepSeek-AI* [и др.]. DeepSeek-V3 Technical Report. — 2025.
6. *AI Index Steering Committee.* Artificial Intelligence Index Report 2026 : тех. отч. / Stanford Institute for Human-Centered Artificial Intelligence (HAI). — 2026. — Accessed: 2026-05-25.
7. *Denil M., Shakibi B., Dinh L., Ranzato M., Freitas N. de.* Predicting Parameters in Deep Learning. — 2014.
8. *Yu M., Wang D., Shan Q., Reed C., Wan A.* The Super Weight in Large Language Models. — 2024.
9. *Dong P., Li L., Liu X., Tang Z., Liu X., Wang Q., Chu X.* LPZero: Language Model Zero-cost Proxy Search from Zero. — 2024.
10. *Tanaka H., Kunin D., Yamins D. L. K., Ganguli S.* Pruning neural networks without any data by iteratively conserving synaptic flow. — 2020.
11. *Lee N., Ajanthan T., Torr P. H. S.* SNIP: Single-shot Network Pruning based on Connection Sensitivity. — 2019.
12. *Lin J., Tang J., Tang H., Yang S., Chen W.-M., Wang W.-C., Xiao G., Dang X., Gan C., Han S.* AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. — 2024.

13. *Dettmers T., Lewis M., Belkada Y., Zettlemoyer L.* LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. — 2022.
14. *Kim S., Hooper C., Gholami A., Dong Z., Li X., Shen S., Mahoney M. W., Keutzer K.* SqueezeLLM: Dense-and-Sparse Quantization. — 2024.
15. *Gromov A., Tirumala K., Shapourian H., Glorioso P., Roberts D. A.* The Unreasonable Ineffectiveness of the Deeper Layers. — 2025.
16. *Song J., Oh K., Kim T., Kim H., Kim Y., Kim J.-J.* SLEB: Streamlining LLMs through Redundancy Verification and Elimination of Transformer Blocks. — 2024.
17. *Sun M., Liu Z., Bair A., Kolter J. Z.* A Simple and Effective Pruning Approach for Large Language Models. — 2024.
18. *Allen-Zhu Z., Li Y., Song Z.* A Convergence Theory for Deep Learning via Over-Parameterization. — 2019.
19. *Hu E. J., Shen Y., Wallis P., Allen-Zhu Z., Li Y., Wang S., Wang L., Chen W.* LoRA: Low-Rank Adaptation of Large Language Models. — 2021.
20. *Tugaryov A., Trutnev A., Demidovskij A., Salnikov I.* ALOE: Boosting Large Language Model Fine-Tuning with Aggressive Loss-Based Elimination of Samples //. — 10.2024.
21. *Dettmers T., Pagnoni A., Holtzman A., Zettlemoyer L.* QLoRA: Efficient Finetuning of Quantized LLMs. — 2023.
22. *Cheng H., Zhang M., Shi J. Q.* A Survey on Deep Neural Network Pruning-Taxonomy, Comparison, Analysis, and Recommendations. — 2024.
23. *Malladi S., Gao T., Nichani E., Damian A., Lee J. D., Chen D., Arora S.* Fine-Tuning Language Models with Just Forward Passes. — 2024.
24. *Hinton G.* The Forward-Forward Algorithm: Some Preliminary Investigations. — 2022.

25. *Cazenavette G., Wang T., Torralba A., Efros A. A., Zhu J.-Y.* Dataset Distillation by Matching Training Trajectories. — 2022.
26. *Shim J.-h., Kong K., Kang S.-J.* Core-set Sampling for Efficient Neural Architecture Search. — 2021.
27. *Jiang A. H.* [и др.]. Accelerating Deep Learning by Focusing on the Biggest Losers. — 2019.
28. *Johnson T. B., Guestrin C.* Training Deep Models Faster with Robust, Approximate Importance Sampling // Advances in Neural Information Processing Systems. T. 31. — Curran Associates, Inc., 2018.
29. *Han Z., Gao C., Liu J., Zhang J., Zhang S. Q.* Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey. — 2024.
30. *Kim J., Lee J. H., Kim S., Park J., Yoo K. M., Kwon S. J., Lee D.* Memory-Efficient Fine-Tuning of Compressed Large Language Models via sub-4-bit Integer Quantization. — 2023.
31. *Kwon S. J., Kim J., Bae J., Yoo K. M., Kim J.-H., Park B., Kim B., Ha J.-W., Sung N., Lee D.* AlphaTuning: Quantization-Aware Parameter-Efficient Adaptation of Large-Scale Pre-Trained Language Models. — 2022.
32. *Xu P., Shao W., Chen M., Tang S., Zhang K., Gao P., An F., Qiao Y., Luo P.* BESA: Pruning Large Language Models with Blockwise Parameter-Efficient Sparsity Allocation. — 2024.
33. *Lin B.* [и др.]. Efficient GPU Kernels for N:M-Sparse Weights in Deep Learning // Proceedings of Machine Learning and Systems. T. 5. — Curan, 2023. — C. 513—525.
34. *Bambhaniya A. R., Yazdanbakhsh A., Subramanian S., Kao S.-C., Agrawal S., Evci U., Krishna T.* Progressive Gradient Flow for Robust N:M Sparsity Training in Transformers. — 2024.

35. *Cheng H., Zhang M., Shi J. Q.* A Survey on Deep Neural Network Pruning-Taxonomy, Comparison, Analysis, and Recommendations. — 2024.
36. *Ma X., Fang G., Wang X.* LLM-Pruner: On the Structural Pruning of Large Language Models. — 2023.
37. *Huang W., Hu Y., Jian G., Zhu J., Chen J.* Pruning Large Language Models with Semi-Structural Adaptive Sparse Training. — 2024.
38. *Yu P., Wang J., Sui X., Ling N., Wang W., Jiang W.* Efficient Post-Training Pruning of Large Language Models with Statistical Correction. — 2026.
39. *Alanova S., Kazistova K., Galaeva E., Kostromina A., Smirnov V., Dmitry R., Dontsov A., Zhelnin M., Burnaev E., Shvetsov E.* Motivating Next-Gen Accelerators with Flexible (N:M) Activation Sparsity via Benchmarking Lightweight Post-Training Sparsification Approaches. — 2026.
40. *Sun Y., Zhang Q., Yu Z., Zhang X., Shen L., Tao D.* MaskPro: Linear-Space Probabilistic Learning for Strict (N:M)-Sparsity on LLMs. — 2026.
41. *Spall J.* Multivariate stochastic approximation using a simultaneous perturbation gradient approximation // IEEE Transactions on Automatic Control. — New York, NY, 1992. — T. 37, № 3. — C. 332—341.
42. *Brown R.* Donald O. Hebb and the Organization of Behavior: 17 years in the writing // Molecular Brain. — 2020. — Дек. — T. 13.
43. *Demidovskij A., Tugaryov A., Salnikov I., Frolova O., Trutnev A., Xie P., Novikova I., Zharikov E., Blyudova V., Ignatiev Y.* Hebb-Inspired Low Rank Adapters for Large Language Models Fine-Tuning // PRICAI 2025: Trends in Artificial Intelligence. — Singapore : Springer Nature Singapore, 2026. — C. 603—612.
44. *Liu S.-Y., Wang C.-Y., Yin H., Molchanov P., Wang Y.-C. F., Cheng K.-T., Chen M.-H.* DoRA: Weight-Decomposed Low-Rank Adaptation. — 2024.

45. *Meng F., Wang Z., Zhang M.* PiSSA: Principal Singular Values and Singular Vectors Adaptation of Large Language Models. — 2025.
46. *Zhang Q., Chen M., Bukharin A., Karampatziakis N., He P., Cheng Y., Chen W., Zhao T.* AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. — 2023.
47. *Demidovskij A. V., Salnikov I. G., Tugaryov A. M. [и др.].* Comprehensive Weight Decomposition Analysis of Modern Parameter-Efficient Methods // Optical Memory and Neural Networks. — 2024. — T. 33, Suppl 3. — S513—S522.
48. *Frankle J., Carbin M.* The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. — 2019.
49. *Chen T., Frankle J., Chang S., Liu S., Zhang Y., Wang Z., Carbin M.* The Lottery Ticket Hypothesis for Pre-trained BERT Networks. — 2020.
50. *Han S., Pool J., Tran J., Dally W. J.* Learning both Weights and Connections for Efficient Neural Networks. — 2015.
51. *Kovaleva O., Kulshreshtha S., Rogers A., Rumshisky A.* BERT Busters: Outlier Dimensions that Disrupt Transformers. — 2021.
52. *Zhang C., Bengio S., Singer Y.* Are All Layers Created Equal? — 2022.
53. *Voita E., Talbot D., Moiseev F., Sennrich R., Titov I.* Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. — 2019.
54. *Liu H., Dai Z., So D. R., Le Q. V.* Pay Attention to MLPs. — 2021.
55. *Tyukin G., Dovonon G. J.-S., Kaddour J., Minervini P.* Attention Is All You Need But You Don't Need All Of It For Inference of Large Language Models. — 2024.

56. *He S., Sun G., Shen Z., Li A.* What Matters in Transformers? Not All Attention is Needed. — 2024.
57. *Tian Y., Wang Y., Zhang Z., Chen B., Du S.* JoMA: Demystifying Multilayer Transformers via JOint Dynamics of MLP and Attention. — 2024.
58. *Men X., Xu M., Zhang Q., Wang B., Lin H., Lu Y., Han X., Chen W.* ShortGPT: Layers in Large Language Models are More Redundant Than You Expect. — 2024.
59. *Li P., Yin L., Gao X., Liu S.* Outlier-weighted Layerwise Sampling for LLM Fine-tuning. — 2025.
60. *Kim B.-K., Kim G., Kim T.-H., Castells T., Choi S., Shin J., Song H.-K.* Shortened LLaMA: Depth Pruning for Large Language Models with Comparison of Retraining Methods. — 2024.
61. *Zhou Q., Sheng K., Zheng X., Li K., Sun X., Tian Y., Chen J., Ji R.* Training-free Transformer Architecture Search. — 2022.
62. *An Y., Zhao X., Yu T., Tang M., Wang J.* Systematic Outliers in Large Language Models. — 2025.
63. *Han S., Mao H., Dally W. J.* Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. — 2016.
64. *Xiao G., Lin J., Seznec M., Wu H., Demouth J., Han S.* SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. — 2024.
65. *Kim S., Hooper C., Gholami A., Dong Z., Li X., Shen S., Mahoney M. W., Keutzer K.* SqueezeLLM: Dense-and-Sparse Quantization. — 2024.
66. *Qin X., Wang Z.* NASNet: A Neuron Attention Stage-by-Stage Net for Single Image Deraining. — 2020.
67. *Chen M., Peng H., Fu J., Ling H.* AutoFormer: Searching Transformers for Visual Recognition. — 2021.

68. *Su X., You S., Xie J., Zheng M., Wang F., Qian C., Zhang C., Wang X., Xu C.* ViTAS: Vision Transformer Architecture Search. — 2021.
69. *Demidovskij A., Novikova I., Tugaryov A., Blyudova V., Frolova O., Ignatiev Y., Salnikov I., Trutnev A., Zharikov E.* On the Influence of Layer Importance on LLM Fine-Tuning Acceleration and Quality // ECAI 2025: 27th European Conference on Artificial Intelligence. T. 413. — IOS Press, 2025. — C. 4233—4240.
70. *Новикова И. А.* novikova_i_a_uskorenie-processa-doobucheniya-s-pomoshchyu-analiza-vajnosti-sloev-bolshih-yazykovyh-modeley. — 2025.
71. *Mi Z., Kong Z., Yuan G., Huang S.* ACE: Exploring Activation Cosine Similarity and Variance for Accurate and Calibration-Efficient LLM Pruning. — 2025.
72. *Autar D.* daryl149/llama-2-7b-hf. Hugging Face — [huggingface.co](https://huggingface.co/daryl149/llama-2-7b-hf). — 2023 ; — [Accessed 24-05-2026]. <https://huggingface.co/daryl149/llama-2-7b-hf>.
73. *Autar D.* daryl149/llama-2-13b-chat-hf. Hugging Face — [huggingface.co](https://huggingface.co/daryl149/llama-2-13b-chat-hf). — 2023 ; — [Accessed 24-05-2026]. <https://huggingface.co/daryl149/llama-2-13b-chat-hf>.
74. *Qwen.* Qwen/Qwen2.5-3B. Hugging Face — [huggingface.co](https://huggingface.co/Qwen/Qwen2.5-3B). — 2024 ; — [Accessed 24-05-2026]. <https://huggingface.co/Qwen/Qwen2.5-3B>.
75. *Qwen.* Qwen/Qwen2.5-7B. Hugging Face — [huggingface.co](https://huggingface.co/Qwen/Qwen2.5-7B). — 2024 ; — [Accessed 24-05-2026]. <https://huggingface.co/Qwen/Qwen2.5-7B>.
76. *Qwen.* Qwen/Qwen2.5-32B. Hugging Face — [huggingface.co](https://huggingface.co/Qwen/Qwen2.5-32B). — 2024 ; — [Accessed 24-05-2026]. <https://huggingface.co/Qwen/Qwen2.5-32B>.
77. *Mangrulkar S., Gugger S., Debut L., Belkada Y., Paul S., Bossan B.* PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. — 2022. — <https://github.com/huggingface/peft>.

78. *Bisk Y., Zellers R., Bras R. L., Gao J., Choi Y.* PIQA: Reasoning about Physical Commonsense in Natural Language. — 2019.
79. *Zellers R., Holtzman A., Bisk Y., Farhadi A., Choi Y.* HellaSwag: Can a Machine Really Finish Your Sentence? — 2019.
80. *Sakaguchi K., Bras R. L., Bhagavatula C., Choi Y.* WinoGrande: An Adversarial Winograd Schema Challenge at Scale. — 2019.
81. *Clark P., Cowhey I., Etzioni O., Khot T., Sabharwal A., Schoenick C., Tafjord O.* Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. — 2018.
82. *Mihaylov T., Clark P., Khot T., Sabharwal A.* Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. — 2018.
83. *Hendrycks D., Burns C., Basart S., Zou A., Mazeika M., Song D., Steinhardt J.* Measuring Massive Multitask Language Understanding. — 2021.
84. *Gao L.* [и др.]. A framework for few-shot language model evaluation. — Бep. v0.4.3. — 07.2024.
85. *Merity S., Xiong C., Bradbury J., Socher R.* Pointer Sentinel Mixture Models. — 2016.
86. *Gou J., Yu B., Maybank S. J., Tao D.* Knowledge Distillation: A Survey // International Journal of Computer Vision. — 2021. — Март. — Т. 129, № 6. — С. 1789—1819.
87. *Malladi S., Gao T., Nichani E., Damian A., Lee J. D., Chen D., Arora S.* Fine-Tuning Language Models with Just Forward Passes. — 2024.
88. *Liu Z., Oguz B., Zhao C., Chang E., Stock P., Mehdad Y., Shi Y., Krishnamoorthi R., Chandra V.* LLM-QAT: Data-Free Quantization Aware Training for Large Language Models. — 2023.

89. *Xu L., Xie H., Qin S.-Z. J., Tao X., Wang F. L.* Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment. — 2023.
90. *Wei J., Wang X., Schuurmans D., Bosma M., Ichter B., Xia F., Chi E., Le Q., Zhou D.* Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. — 2023.
91. *Li X.* [и др.]. FLM-101B: An Open LLM and How to Train It with \$100K Budget. — 2025.
92. *lv K., Yang Y., Liu T., Gao Q., Guo Q., Qiu X.* Full Parameter Fine-tuning for Large Language Models with Limited Resources. — 2024.
93. *Zhang Y.* [и др.]. Revisiting Zeroth-Order Optimization for Memory-Efficient LLM Fine-Tuning: A Benchmark. — 2024.
94. *Li P., Yin L., Gao X., Liu S.* Outlier-weighted Layerwise Sampling for LLM Fine-tuning. — 2025.
95. *Zhou A., Ma Y., Zhu J., Liu J., Zhang Z., Yuan K., Sun W., Li H.* Learning N:M Fine-grained Structured Sparse Neural Networks From Scratch. — 2021.
96. *Katharopoulos A., Fleuret F.* Biased Importance Sampling for Deep Neural Network Training. — 2017.
97. *Lagani G., Falchi F., Gennaro C., Amato G.* Hebbian semi-supervised learning in a sample efficiency setting // *Neural Networks*. — 2021. — Ноябрь. — Т. 143. — С. 719—731.
98. *Krithivasan S., Sen S., Venkataramani S., Raghunathan A.* Accelerating DNN Training Through Selective Localized Learning // *Frontiers in Neuroscience*. — 2022. — ЯНВ. — Т. 15.
99. *He J., Wu S., Wen W., Xue C. J., Li Q.* CHESS: Optimizing LLM Inference via Channel-Wise Thresholding and Selective Sparsification. — 2024.

100. *Chen X., Hu Y., Zhang J., Wang Y., Li C., Chen H.* Streamlining Redundant Layers to Compress Large Language Models. — 2025.
101. *Yin L.* [и др.]. Outlier Weighed Layerwise Sparsity (OWL): A Missing Secret Sauce for Pruning LLMs to High Sparsity. — 2024.
102. *Pan R., Liu X., Diao S., Pi R., Zhang J., Han C., Zhang T.* LISA: Layerwise Importance Sampling for Memory-Efficient Large Language Model Fine-Tuning. — 2024.
103. *Cottier B., Rahman R., Fattorini L., Maslej N., Besiroglu T., Owen D.* The rising costs of training frontier AI models. — 2025.
104. *Liu Y., Singh A., Freeman C. D., Co-Reyes J. D., Liu P. J.* Improving Large Language Model Fine-tuning for Solving Math Problems. — 2023.
105. *Maslej N.* [и др.]. Artificial Intelligence Index Report 2025. — 2025.
106. *Zhou A., Ma Y., Zhu J., Liu J., Zhang Z., Yuan K., Sun W., Li H.* Learning N:M Fine-grained Structured Sparse Neural Networks From Scratch. — 2021.
107. *Gromov A., Tirumala K., Shapourian H., Glorioso P., Roberts D. A.* The Unreasonable Ineffectiveness of the Deeper Layers. — 2025 ; — [Accessed 24-05-2026]. <https://github.com/arcee-ai/PruneMe>.
108. *Song J., Oh K., Kim T., Kim H., Kim Y., Kim J.-J.* SLEB: Streamlining LLMs through Redundancy Verification and Elimination of Transformer Blocks. — 2025 ; — [Accessed 24-05-2026]. <https://github.com/jiwonsong-dev/SLEB>.
109. *Kim B.-K., Kim G., Kim T.-H., Castells T., Choi S., Shin J., Song H.-K.* Shortened LLaMA: Depth Pruning for Large Language Models with Comparison of Retraining Methods. — 2024 ; — [Accessed 24-05-2026]. <https://github.com/Nota-NetsPresso/shortened-llm>.

110. *Pan R., Liu X., Diao S., Pi R., Zhang J., Han C., Zhang T.* LISA: Layerwise Importance Sampling for Memory-Efficient Large Language Model Fine-Tuning. — 2024 ; — [Accessed 24-05-2026]. <https://github.com/osehmathias/lisa>.
111. *Li P., Yin L., Gao X., Liu S.* Outlier-weighted Layerwise Sampling for LLM Fine-tuning. — 2024 ; — [Accessed 24-05-2026]. <https://github.com/pixeli99/OWS>.
112. *Draganov A., Vadgama S., Bekkers E. J.* The Hidden Pitfalls of the Cosine Similarity Loss. — 2024.
113. *Arora S., Cohen N., Golowich N., Hu W.* A Convergence Analysis of Gradient Descent for Deep Linear Neural Networks. — 2019.
114. *Liu Y., Agarwal S., Venkataraman S.* AutoFreeze: Automatically Freezing Model Blocks to Accelerate Fine-tuning. — 2021.